LEÇON 2 : Logique Propositionnelle et Constructivisme

Connecteurs ET, OU et Négation

Pablo Donato

24/03/2026

Après avoir exploré l'implication ($\rightarrow$), notre voyage au cœur de la correspondance de Curry-Howard se poursuit. Aujourd'hui, nous allons découvrir comment Rocq traite la Conjonction (ET), la Disjonction (OU), et l'Absurdité (NON).

Nous adopterons une perspective profondément constructiviste (la sémantique Brouwer-Heyting-Kolmogorov, ou BHK), où chaque connecteur logique n'est pas une simple table de vérité, mais une véritable **structure de données** informatique construite par nos soins !

```
Notation "( x , y )" := (conj x y).
Notation "'ι₁'" := or_introl (at level 0).
Notation "'ι₂'" := or_intror (at level 0).

Section Lesson2.

Variables A B C : Prop.
```

1 La Conjonction (Le ET : $\wedge$)

En logique de la réalisation (constructivisme), posséder une preuve formelle de $A \wedge B$, c'est posséder indépendamment une preuve de A ET une preuve de B. Informatiquement, cela correspond exactement à la notion de **paire** (ou "tuple").

La règle d'introduction de la conjonction en déduction naturelle est la suivante :

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B}{\Gamma \vdash (M, N) : A \wedge B} \wedge I$$

Pour construire/introduire une telle paire dans Rocq, nous utilisons la tactique `split`.

```
Lemma ET_intro : A -> B -> A /\ B.
Proof.
  intros x y.
  split.
  - exact x.
  - exact y.
Qed.

Print ET_intro.
```

Remarque 1

Observons le terme final. Le compilateur nous montre que le terme de preuve est bel et bien structuré comme une paire !

À l'inverse, si nous *avons* une hypothèse de type $A \wedge B$, nous pouvons la "détruire" pour en extraire les deux composants sous-jacents, grâce à la commande `destruct`. Cela correspond aux **projections** π_1 et π_2 :

$$\frac{\Gamma \vdash M : A \wedge B}{\Gamma \vdash \pi_1 M : A} \wedge E_1 \qquad \frac{\Gamma \vdash M : A \wedge B}{\Gamma \vdash \pi_2 M : B} \wedge E_2$$

```
Lemma ET_symetrie : A /\ B -> B /\ A.
Proof.
  intros H_ab.
  destruct H_ab as [x y].
  split.
  - exact y.
  - exact x.
Qed.
```

2 La Disjonction (Le OU : $\vee$)

Voici le grand point de rupture du constructivisme ! Pour prouver $A \vee B$, vous ne pouvez pas joyeusement affirmer "bon, l'un des deux doit être vrai", sans savoir lequel. Vous DEVEZ exhiber consciemment une preuve ou bien de A, ou bien de B.

Informatiquement, c'est ce qu'on appelle un **type somme**. L'ordinateur conserve la trace indélébile du choix effectué via des fonctions d'injection : `left` correspond à ι_1 et `right` correspond à ι_2.

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \iota_1 M : A \vee B} \vee I_1 \qquad \frac{\Gamma \vdash N : B}{\Gamma \vdash \iota_2 N : A \vee B} \vee I_2$$

```
Lemma OU_intro_gauche : A -> A \/ B.
Proof.
  intros x.
  left.
  exact x.
Qed.

(** On constate que le terme de preuve final est l'application de l'
   injection
    gauche [ι1] à la variable [x] de type [A].
    On note aussi que le type [B] est passé en argument juste avant [x]
       : c'est
    parce que Rocq a besoin de stocker le type de la branche inutilisée
       ... *)
Print OU_intro_gauche.
```

Remarque 2: Structures de Données vs Contrôle

En informatique, on sépare généralement deux grands piliers :

1. **Structures de Données** : comment la machine représente l'information (ex : les paires pour le ET, ou les injections pour le OU).
2. **Structures de Contrôle** : comment la machine prend des décisions et oriente le flux du calcul (ex : le `if... then... else...`).

Posséder une disjonction $A \vee B$ en hypothèse forme une structure de donnée **fermée**, car on ne sait pas a priori quel choix a été fait (injection gauche ou droite). Pour l'exploiter,

nous avons besoin d'une structure de contrôle pour "ouvrir" cette boîte. La généralisation du `if` s'appelle le **Filtrage par Motif** (`match ... with`).
C'est un aiguillage : "SI la boîte contient une preuve de A, ALORS applique ce raisonnement ; SI elle contient une preuve de B, ALORS applique cet autre raisonnement". C'est ce que génère la tactique `destruct`.

```
Lemma OU_symetrie : A \/ B -> B \/ A.
Proof.
  intros H_ou.
  destruct H_ou as [x | y].
  - (* Cas 1 : La preuve d'origine etait une injection gauche, pour A *)
    right. exact x.
  - (* Cas 2 : La preuve d'origine etait une injection droite, pour B *)
    left. exact y.
Qed.
```

3 L'Absurdité et la Négation

Dans un monde constructiviste, la définition de la négation est d'une élégance presque poétique : $\neg A$ est exactement une **abréviation** pour $A \to \bot$ (où $\bot$ dénote l'absurdité, `False` dans Rocq). Prouver que A est faux, c'est écrire une fonction qui, si elle recevait une preuve de A, réussirait à produire l'absurdité.

Et qu'est-ce que `False` ? C'est le type ultimement vide : il n'a **aucun habitant**.

```
Lemma non_contradiction : ~(A /\ ~A).
Proof.
  intros H_impossible.
  destruct H_impossible as [a non_a].
  apply non_a.
  exact a.
Qed.
```

Le Principe d'Explosion (*Ex Falso Quodlibet*)

Si vous obtenez une preuve de `False` dans votre contexte, c'est que vous avez une contradiction. Et d'une contradiction, vous pouvez déduire N'IMPORTE QUOI. En Rocq, c'est la tactique `exfalso`.

$$\frac{\Gamma \vdash M : \bot}{\Gamma \vdash \text{abort } M : A} \bot E$$

```
Lemma explosion : False -> A.
Proof.
  intro f.
  exfalso.
  exact f.
Qed.
```

4 La Crise du Constructivisme et le Tiers-Exclu

L'idéalisme intuitionniste a un revers : le Tiers-Exclu $A \vee \neg A$ n'est pas prouvable de manière générique. Réussir à le prouver reviendrait à posséder l'Algorithme Universel capable de trancher

la vérité de n'importe quelle proposition sans calcul. Ce rêve de David Hilbert a été ruiné par Gödel et Turing.

```
Lemma tiers_exclu_fail : A \/ ~A.
Proof.
  left. Undo.
  right. intro H.
Abort.
```

Pour retrouver la logique classique, on doit imposer le Tiers-Exclu comme une vérité infuse via la commande `Axiom`.

```
Axiom LEM : forall X : Prop, X \/ ~X.

Lemma double_negation : ~~A -> A.
Proof.
  intros non_non_a.
  destruct (LEM A) as [x | non_a].
  - exact x.
  - exfalso. apply non_non_a. exact non_a.
Qed.
```

5 Exercices à faire

Exercice 1: Lois de De Morgan (Sens Intuitionniste)

Prouver formellement $(A \vee B) \to \neg(\neg A \wedge \neg B)$. Ce sens est constructif, nul besoin du Tiers-Exclu !

Exercice 2: Lois de De Morgan (Sens Classique)

Prouver la réciproque $\neg(\neg A \wedge \neg B) \to (A \vee B)$. Attention : ici le constructivisme est paralysé, vous devrez utiliser l'axiome `LEM`.

```
End Lesson2.
```